

Conceptos Fundamentales de Arquitectura de Software

Juan Sebastian Perez Galvis

Código: 202413228925

Ingeniería de Sistemas · Facultad de Ingenierías

Universidad Simón Bolívar

Curso: Arquitectura de Software · Código 5111

Docente: Oswaldo Enrique Laguado Aguilar

23 de agosto de 2026

Introducción

El presente informe desarrolla la actividad de la primera unidad del curso de Arquitectura de Software, dedicada a los conceptos fundamentales de la materia y al uso de la inteligencia artificial como apoyo y no como sustituto del razonamiento. La guía plantea que una arquitectura no es un dibujo sino una cadena de decisiones conscientes, y que en esta materia lo que se evalúa no es la respuesta de la herramienta sino la calidad del encargo y la finura del análisis con que se compara (Universidad Simón Bolívar, 2026).

El documento se organiza en tres bloques. El ejercicio de manejo del prompt, la sección 4, muestra cómo un encargo bien formulado cambia el resultado frente a uno redactado con prisa. Las preguntas individuales, la sección 6, recorren los seis conceptos de la unidad y los encadenan: los atributos de calidad condicionan la arquitectura, esta se concreta en capas y niveles, y el patrón Modelo-Vista-Controlador ordena el interior de una de esas capas. Los puntos en pareja, la sección 7, contrastan dos definiciones de arquitectura y defienden una decisión de diseño reconociendo sus renuncias. Todo se apoya en libros físicos del Sistema de Bibliotecas de la Universidad, cuya portada y contraportada se anexan al final, tal como pide la actividad de biblioteca.

4 · Ejercicio de Manejo del Prompt

La sección 4 de la guía propone un ejercicio doble: escribir un prompt propio sobre un tema del curso y demostrar que se entiende por qué uno funciona y el otro no. Lo que se califica, dice la guía, no es la respuesta de la herramienta sino la calidad del prompt redactado y la finura del análisis con que se compara (Universidad Simón Bolívar, 2026). Este documento desarrolla el ejercicio completo con el encargo A, en el orden que pide la sección 4.2.

4.1 · Encargo Elegido

a) El Encargo Elegido. Opción A: explicar por qué en una aplicación web se separa el servidor web del servidor de base de datos, y qué implica esa separación para la conectividad entre ambos.

Se escogió esta opción porque aborda la primera decisión de despliegue que se enfrenta al pasar de una aplicación de clase a un entorno de producción, y porque la segunda parte del enunciado, la conectividad, obliga a decir lo que un prompt vago suele pasar por alto.

4.2 · Qué Se Entrega

b) El Prompt Malo. Transcrito literal, como lo escribiría un compañero con prisa: una línea, sin contexto y sin formato.

explícame por qué en una aplicación web el servidor web y el servidor de base de datos van separados

No es absurdo a propósito: es el encargo que uno manda a las once de la noche, y por eso sirve como contraste. Si el prompt malo fuera deliberadamente disparatado, el análisis no valdría nada.

c) El Prompt Bueno. Transcrito literal, con los seis elementos marcados entre corchetes.

[ROL] Actúas como un ingeniero de software con experiencia en despliegue y operación de aplicaciones web.

[TAREA] Explicame por qué en una aplicación web se separa el servidor web del servidor de base de datos y qué implica esa separación para la conectividad entre ambos.

[CONTEXTO] Soy estudiante de séptimo semestre de Ingeniería de Sistemas. He hecho aplicaciones web que corren en un solo servidor, con la base de datos en el mismo equipo, y no entiendo por qué en entornos de producción los separan.

[FORMATO] Empieza con una respuesta directa de máximo tres líneas. Después una tabla de tres columnas: qué cambia con la separación, qué gana, qué cuesta. Termina explicando el papel de la conexión entre ambos servidores.

[RESTRICCIONES] No nombres tecnologías comerciales concretas. Máximo 300 palabras. Distingue entre lo que es consenso de la disciplina y lo que es tu interpretación, marcando con «(mi interpretación)».

[VERIFICACIÓN] Cierra diciendo qué afirmaciones debería yo confirmar en un libro antes de darlas por buenas y qué información adicional necesitarías tú para responder con más precisión.

d) Las Dos Respuestas, Recortadas. Ambas se generaron con la herramienta opencode, modelo deepseek-v4-flash, el 23 de agosto de 2026. Se aplicó la regla del recorte: máximo 200 palabras por respuesta, el texto va tal como salió, los cortes se marcan con [...], se conserva el inicio y el fragmento que se comenta, y las respuestas se separan de la redacción propia en bloques de cita, con la misma fuente del documento (Universidad Simón Bolívar, 2026).

Respuesta al prompt malo. Recorte de 63 palabras.

Claro. Te lo explico simple: un servidor web y un servidor de base de datos van separados por razones de escalabilidad, seguridad y rendimiento. [...] En definitiva, separarlos te da independencia, seguridad y la posibilidad de crecer de forma flexible según lo que necesite cada parte. ¿Quieres que te explique cómo se conectan en un caso real como Node.js, Python o el que uses?

Respuesta al prompt bueno. Recorte de 136 palabras. Se omitió la tabla de tres columnas que la respuesta incluyó en el tramo central.

Tu intuición es correcta: en un solo servidor todo se conecta por memoria local, pero en producción se separan por arquitectura, no por capricho. La separación convierte la base de datos en un recurso independiente que crece, se protege y se escala por sí solo. Eso exige que la conexión entre ambos deje de ser gratuita y pase a ser una parte del sistema que hay que diseñar. [...] La conexión es el punto crítico de la separación: es lo que une dos procesos que ya no comparten memoria. [...] También define cuántas consultas simultáneas soporta la aplicación: si se duplica el servidor web pero la base recibe todas las conexiones, se convierte en el cuello de botella. [...] Para responderte con más precisión necesitaría saber si te interesa el porqué de negocio (costo, disponibilidad) o el cómo técnico.

E) Tabla Comparativa de las Dos Respuestas.

Tabla 1

Comparación entre la Respuesta al Prompt Malo y la Respuesta al Prompt Bueno

Criterio	Respuesta al prompt malo	Respuesta al prompt bueno
¿Responde al encargo o divaga	Responde, pero como lista de beneficios genéricos	Responde a las dos partes. Primero el porqué de la separación y luego

Criterio	Respuesta al prompt malo	Respuesta al prompt bueno
hacia generalidades?	(escalabilidad, seguridad, rendimiento). La segunda mitad del encargo, la conectividad, no aparece en ningún momento.	la conexión como punto crítico: «es lo que une dos procesos que ya no comparten memoria».
¿Sirve para tomar alguna decisión concreta?	No. Queda en divulgación; no indica cuándo conviene separar ni qué se compromete al hacerlo.	Sí. Da criterios accionables: la conexión hay que diseñarla (dirección, puerto, autenticación, cifrado) y advierte del cuello de botella cuando la base recibe todas las conexiones simultáneas.
¿Tuvo en cuenta restricciones del contexto?	No había contexto que considerar. El prompt no lo daba y la herramienta respondió como si el caso fuera cualquiera.	Sí. Partió del caso declarado («en un solo servidor todo se conecta por memoria local») y acotó la respuesta al entorno de producción.
¿Reconoce algún límite o incertidumbre?	No. Afirma con certeza y no marca ninguna afirmación como opinión.	Sí. Marcó «(mi interpretación)» en dos puntos, pidió confirmar conceptos en un libro y preguntó qué información le falta.
¿El formato es aprovechable tal como llegó?	No. Es un párrafo corrido con una metáfora al inicio; hay que releerlo para extraer los motivos.	En parte. Llegó estructurado: frase directa, tabla de tres columnas (omitida en el recorte por el límite) y cierre. La tabla llegó como texto plano con barras, de modo que hubo que normalizarla.

Criterio	Respuesta al prompt malo	Respuesta al prompt bueno
¿Qué habría que verificar antes de citarla?	Todo, porque no distingue consenso de opinión y no da pistas para confirmar ninguna afirmación.	Los puntos que la propia respuesta marcó como interpretación y los conceptos que pidió confirmar en un libro: conexión, agrupación de conexiones e impacto de la latencia.

Nota. Elaboración propia a partir de las respuestas recortadas en el literal d.

F) Análisis por Elemento del Prompt.

Tabla 2

Efecto de Cada Elemento del Prompt Bueno sobre la Respuesta

Elemento	Qué se agregó en el prompt bueno	Qué cambió en la respuesta
Rol	Ingeniero de software con experiencia en despliegue y operación.	La respuesta subió de registro: habla de memoria local, de recursos que crecen y se escalan, de conexiones y de cuello de botella. Sin rol, la respuesta al prompt malo se quedó en divulgación, con la metáfora del restaurante.
Tarea	Dos verbos con sus objetos: el porqué de la separación y lo que implica para la conectividad entre ambos.	Es el cambio de fondo. La respuesta dejó de ser una lista de ventajas y convirtió la conexión entre servidores en el eje del texto. Con el prompt malo ese tema no existía.

Elemento	Qué se agregó en el prompt bueno	Qué cambió en la respuesta
Contexto	Estudiante de séptimo semestre, con aplicaciones que corren en un solo servidor y que no entiende por qué en producción los separan.	La respuesta partió del punto de partida declarado en el contexto («en un solo servidor todo se conecta por memoria local») y no gastó espacio en lo que el estudiante ya sabía.
Formato	Respuesta directa de máximo tres líneas, luego una tabla de tres columnas y un cierre con el papel de la conexión.	La respuesta llegó con estructura: abrió con una frase directa y, en el tramo que el recorte omitió, entregó la tabla pedida. Sin esa petición, llegaba un párrafo corrido.
Restricciones	No nombrar tecnologías comerciales, máximo 300 palabras y marcar con «(mi interpretación)» lo que no sea consenso.	La respuesta no propuso productos concretos y etiquetó los puntos que eran interpretación suya. La respuesta al prompt malo terminó ofreciendo Node.js o Python, es decir, tecnología comercial.
Verificación	Cerrar indicando qué debería confirmarse en un libro y qué información le falta a la herramienta.	La respuesta cerró pidiendo información faltante (el porqué de negocio o el cómo técnico) y qué confirmar en un libro, en lugar de despedirse con una oferta de seguir hablando, que fue lo que hizo la respuesta al prompt malo.

Nota. Elaboración propia.

El elemento que produjo el mayor cambio fue la tarea: la pregunta del prompt malo era solo «por qué van separados» y la herramienta la leyó como una invitación a listar bondades,

mientras que al añadir «y qué implica esa separación para la conectividad entre ambos» el encargo se volvió concreto y la respuesta tuvo que tratar la conexión como objeto de explicación. Ese segundo verbo cambió el tema del texto y no su tono, y por eso pesó más que los demás.

El elemento que no cambió prácticamente nada fue el rol, porque el fondo ya lo aportaba la tarea: el tema es un lugar común de la disciplina y la herramienta lo desarrolla bien sin necesidad de una persona, aunque la respuesta buena asumió un tono más de operación. La diferencia entre las dos respuestas se explica por la tarea, el contexto y la verificación, mientras que el rol solo afinó el estilo. La guía advierte que no todos los elementos pesan igual en cada encargo, y en este caso fue el que menos pesó.

G) Cierre Reflexivo.

La próxima vez se pedirán el formato y las restricciones desde el primer mensaje y no se aceptará la primera respuesta como definitiva, pues se la tratará como hipótesis y se le indicará a la herramienta qué estuvo mal para iterar. Se escribirá el contexto aunque parezca que sobra, porque aquí fue lo que ancló la respuesta al caso concreto, y tampoco se pedirán referencias, sino pistas para buscarlas. Lo que la herramienta no pudo hacer por el autor fue decidir qué tema escoger, juzgar cuál respuesta servía y sostener ese juicio con argumentos, y eso no se delega: es el criterio que la guía califica.

6 · Preguntas Individuales

La sección 6 de la guía plantea seis preguntas sobre los conceptos fundamentales de la materia, cada una con todos sus literales en orden. El marco de referencia de todas las respuestas es el libro *Fundamentals of Software Architecture* de Richards y Ford (2020), cuyos apartados se transcriben tal como aparecen en el original allí donde la guía exige sustentar con una fuente (Universidad Simón Bolívar, 2026). El resto de los apartados se desarrolla sobre ese marco, sin repetir la autoría en cada párrafo.

Pregunta 1 · Arquitectura de Software: La Frontera

a) La Definición del Autor. La arquitectura de software se define en el libro como el conjunto que resulta de combinar la estructura del sistema con las condiciones que este debe soportar: «software architecture consists of the structure of the system ... combined with architecture characteristics (“-ilities”) the system must support, architecture decisions, and finally design principles» (Richards y Ford, 2020). Con otras palabras, no es un dibujo ni un documento sino un conjunto de decisiones sobre cuatro ejes: la estructura (el estilo en que se organiza el sistema), las características que debe soportar (los atributos de calidad), las decisiones que fijan sus reglas de construcción, por ejemplo que solo las capas de negocio y de servicios accedan a la base de datos, y los principios de diseño que guían al equipo sin imponerle una regla dura. La propia definición no es estática: toda arquitectura resulta de su contexto, y la misma elección puede ser acertada en un entorno y equivocada en otro.

b) Tabla de Arquitectura, Diseño Detallado e Implementación. Como se muestra en la Tabla 3, cada nivel de trabajo decide cosas distintas y en momentos distintos.

Tabla 3

Qué Decide y Cuándo Se Decide Cada Nivel de Trabajo

Nivel	Qué decide	Cuándo se decide
Arquitectura	Qué estructura adopta el sistema, qué características de calidad debe soportar y qué reglas de construcción rigen su desarrollo.	Al inicio del proyecto, antes de programar: define el marco para el resto de las decisiones.
Diseño detallado	Cómo se organizan los componentes dentro de la arquitectura elegida: módulos, clases y soluciones localizadas.	Durante la planeación de cada módulo o iteración: vive dentro de la arquitectura.
Implementación	Detalles locales del código: algoritmos, nombres, formatos y tecnologías puntuales dentro de las reglas fijadas.	Durante el desarrollo, en cada tarea concreta.

Nota. El diseño vive dentro de la arquitectura y la implementación se guía por las decisiones y los principios fijados, de acuerdo con el marco del literal a.

c) Una Decisión Cara de Revertir y una Barata. Una decisión arquitectónica cara de revertir es elegir que el sistema se construya como un monolito por capas desplegado en una sola unidad junto con su base de datos, porque el estilo por capas escala poco y, cuando el sistema crece, cambiar esa estructura obliga a rediseñar el despliegue y a reescribir código que ya funciona. Una decisión de implementación barata de cambiar es, en cambio, el nombre de una variable o el orden de los campos de un formulario: se modifica en minutos y casi nada depende de ello. Lo que separa a una de la otra no es la dificultad técnica sino cuántas decisiones dependen de cada elección, que es lo que expresa la noción de connascencia: dos componentes están en connascencia si un cambio en uno obliga a modificar al otro para conservar la corrección del sistema. La decisión cara arrastra a muchas otras; la barata, a casi ninguna.

d) Cierre. Un sistema puede estar bien programado y mal arquitecturado porque la calidad del código es local y la arquitectura es sistémica: sus cuatro ejes, la estructura, las características, las decisiones y los principios, se juzgan en el conjunto y no mirando una línea. Un equipo puede escribir clases correctas y eficientes y, a la vez, elegir una estructura que no soporta la escala esperada. El código se ve; la arquitectura se decide, y por eso una buena programación no garantiza una buena arquitectura.

Pregunta 2 · Atributos de Calidad: Lo Que No Se Puede Tener Todo

a) Qué Es un Atributo de Calidad y Su Diferencia con un Requisito Funcional. Las características de arquitectura son los criterios de éxito del sistema: «The architecture characteristics define the success criteria of a system, which is generally orthogonal to the functionality of the system» (Richards y Ford, 2020). Describen, entonces, cómo debe comportarse el sistema y no qué debe hacer, y esa es la diferencia con los requisitos funcionales. Un atributo llega a ser una característica de arquitectura cuando cumple tres condiciones: especifica una consideración de diseño ajena al dominio, influye en algún aspecto estructural del diseño y es crítico para el éxito de la aplicación.

b) Cinco Atributos y la Pregunta para Dimensionarlos. Como se muestra en la Tabla 4, para cada atributo se formula una pregunta que permite al cliente dimensionar su importancia.

Tabla 4

Atributos de Calidad y Preguntas al Cliente para Dimensionar Cada Uno

Atributo	Pregunta al cliente
Escalabilidad	¿Cuántos usuarios simultáneos debe soportar el sistema en la semana más cargada del semestre y cuánto puede crecer esa cifra antes de que el crecimiento sea un problema?

Atributo	Pregunta al cliente
Disponibilidad	¿Cuánto tiempo puede estar caído el sistema en un día de matrículas sin que se considere un incidente grave?
Seguridad	¿Cuál es el dato más sensible del sistema y qué ocurre si se filtra una contraseña o el registro de un estudiante?
Rendimiento	¿Cuál es el tiempo de respuesta aceptable para la consulta más usada y en qué momento un retraso empieza a molestar al usuario?
Mantenibilidad	¿Con qué frecuencia cambiarán las reglas de negocio y qué tan urgente es aplicarlas sin reescribir el sistema?

Nota. Las preguntas se elaboraron sobre la noción de atributo desarrollada en el literal a, aplicada a los atributos operativos de la clasificación del libro.

c) Dos Atributos Que Compiten: Escalabilidad Frente a Simplicidad. Escalabilidad y simplicidad compiten porque casi todo lo que se gana en la primera se paga en la segunda. El estilo por capas es barato, conocido y fácil de mantener, pero al ser un monolito único solo escala hasta cierto punto; para escalar hay que separar responsabilidades en servicios desplegados de manera independiente, y eso introduce infraestructura, red y complejidad operativa que antes no existían. Un caso concreto: el portal de préstamos de la biblioteca universitaria, construido como monolito por capas, recibe el pico de solicitudes del inicio de semestre y su servidor se satura; duplicar los servidores de aplicación no resuelve el problema porque el sistema sigue siendo un solo quantum con una base de datos única como cuello de botella. La salida responsable obliga a repartir el sistema y a sacrificar la simplicidad que hacía barata su operación, porque cada atributo que se gana se paga con complejidad de diseño.

d) Por Qué No Existe «la Mejor Arquitectura» en Abstracto. No existe porque los criterios de éxito dependen del problema y del contexto: no hay respuestas correctas en

arquitectura, solo costosas, y cada característica que se suma añade complejidad al diseño. La misma decisión puede ser un acierto para un sistema y un error para otro, según las restricciones y las renunciaciones que se acepten.

Pregunta 3 · Arquitectura Web: Nivel No Es Lo Mismo Que Capa

a) El Esquema Cliente-servidor. El esquema cliente-servidor separa la funcionalidad técnica entre el frontend y el backend: «A fundamental style in architecture separates technical functionality between frontend and backend, called a two-tier, or client/server, architecture» (Richards y Ford, 2020). El cliente asume la presentación y la interacción con el usuario, mientras que el servidor asume el procesamiento y el cuidado de los datos; en la web moderna el navegador se conecta al servidor web y este al servidor de base de datos, con responsabilidades separadas en cada extremo.

b) Arquitectura de Dos, Tres y N Niveles. En la arquitectura de dos niveles, o cliente-servidor, la presentación corre en el cliente y el resto en el servidor; aunque la base de datos esté físicamente separada, se sigue contando como dos niveles cuando el servidor web y el de datos corren en la misma clase de máquina dentro del centro de cómputo. La de tres niveles agrega una separación más: un nivel de presentación en el navegador, un nivel de aplicación manejado por un servidor de aplicaciones y un nivel de base de datos con un servidor de datos. De ahí se generaliza a n niveles, sumando niveles de despliegue a medida que se separan más tipos de servidor. Para una aplicación web universitaria, el despliegue de tres niveles resulta así: el navegador del estudiante es el nivel de presentación, un servidor web aloja la lógica de negocio y el acceso a datos, y un servidor de base de datos guarda los registros de matrículas, cursos y notas; la comunicación cruza la red entre nivel y nivel, y cada uno se actualiza y se dimensiona por separado.

c) Nivel Frente a Capa. El nivel es una unidad de despliegue físico y la capa, una organización del código. La topología física del estilo en capas admite variantes: una junta presentación, negocio y persistencia en una sola unidad con la base de datos externa; otra separa la presentación en su propia unidad; y una tercera despliega todas las capas, incluida la base embebida, en un único servidor, cosa que conviene para aplicaciones pequeñas. Por eso un sistema puede tener tres capas de código en carpetas separadas y correr entero en un solo servidor: la separación existe en el código y da mantenibilidad, pero no da tolerancia a fallos ni escalabilidad, porque el nivel físico es único. Confundir capa con nivel lleva a dimensionar mal la infraestructura y a asegurarle al cliente que el sistema es robusto por estar «en tres niveles» cuando todo compite por la misma máquina, que es el error que la guía marca como el más frecuente del curso.

d) Escalabilidad Cuando los Datos y la Lógica Comparten Nivel. Cuando la capa de datos y la de lógica comparten el mismo nivel físico y llega un pico de usuarios, el procesamiento y los accesos a datos compiten por la misma CPU, la misma memoria y el mismo disco, y el nivel se satura como un todo: al ser un monolito único, la aplicación solo puede escalar hasta cierto punto y no hay forma de crecer solo el procesamiento o solo los datos. Escalar el nivel exigiría separar la lógica de los datos en niveles independientes, que es exactamente la distinción que se pierde cuando capa y nivel se confunden.

Pregunta 4 · Separación en Capas: Qué Gana y Qué Cuesta

a) Responsabilidades de Cada Capa. Cada capa tiene una responsabilidad definida: «the presentation layer would be responsible for handling all user interface and browser communication logic, whereas the business layer would be responsible for executing specific business rules associated with the request» (Richards y Ford, 2020). A esas dos se suman la capa de persistencia, que resuelve el acceso a los datos, y la de base de datos, que administra el motor; el flujo desciende por las capas y cada una actúa como una abstracción del trabajo necesario para

satisfacer una petición. La presentación no necesita saber cómo se obtienen los datos, porque solo los muestra en pantalla, y la lógica de negocio los recibe de la persistencia para aplicar sus reglas y devolver el resultado hacia arriba.

b) La Operación de Registrar un Préstamo. Registrar el préstamo de un libro en una biblioteca universitaria, con la separación en capas, ocurre en este orden: la presentación recibe del formulario los datos del préstamo (estudiante, ejemplar y fecha) y valida que no vengan vacíos; la lógica de negocio verifica las reglas, que el ejemplar esté disponible, que el estudiante no tenga sanciones y que la política de préstamos permita la entrega; el acceso a datos ejecuta la consulta que registra el préstamo y actualiza la disponibilidad del ejemplar; y la presentación confirma la operación en pantalla. La condición que habilita un préstamo no está en el formulario ni en la consulta SQL sino en la capa de negocio, y si apareciera en cualquiera de esos otros lugares, la separación estaría rota.

c) Dos Desventajas Reales de la Separación en Capas. La separación en capas también se paga, y con dos costos reales. Primero pierde agilidad: la separación por responsabilidades facilita organizar roles, pero un dominio de negocio queda repartido por todas las capas y aplicar un cambio atraviesa varias de ellas, de modo que responder con rapidez cuesta más. Segundo, aparece el anti-patrón del sumidero de arquitectura: muchas peticiones atraviesan todas las capas en simple passthrough, sin lógica de negocio alguna, y cada salto consume tiempo de procesamiento e instanciación de objetos que se desaprovecha, en perjuicio de la memoria y del rendimiento. Ninguna de las dos invalida la separación: la convierten en una decisión con precio y no en un regalo.

d) Cambiar de Interfaz o Cambiar de Motor de Datos. Por el aislamiento de capas, cambiar la interfaz de escritorio a móvil toca únicamente la presentación: se puede reemplazar toda una capa de interfaz sin impactar a las demás mientras los contratos entre capas se mantengan. Cambiar de motor de base de datos, en cambio, toca la capa de acceso a datos y la de

base de datos, porque cambian las consultas, el mapeo y la configuración de conexión, mientras que la lógica de negocio y la presentación permanecen intactas si la interfaz de la capa de acceso no varía. En el primer caso se toca la presentación y el resto no; en el segundo, el acceso a datos y el resto no, y ambas operaciones son posibles precisamente porque cada capa está aislada.

Pregunta 5 · MVC: El Flujo Completo

a) Responsabilidades del Modelo, la Vista y el Controlador. El Modelo-Vista-Controlador es tratado como un patrón de diseño y no como una arquitectura de despliegue: «the Model-View-Controller design pattern matches with this architectural pattern, making it easy for developers to understand» (Richards y Ford, 2020). En ese patrón, el modelo concentra los datos y las reglas que operan sobre ellos, la vista produce lo que el usuario ve a partir de datos ya resueltos, y el controlador recibe la entrada del usuario, la valida y orquesta la interacción entre el modelo y la vista.

b) Recorrido Completo de una Petición. El recorrido comienza cuando el usuario hace clic en consultar el préstamo de un libro: el navegador envía la petición, el enrutador determina qué controlador la atiende, el controlador valida los datos, solicita al modelo la información del préstamo, recibe los objetos ya resueltos y selecciona la vista; la vista recibe únicamente esos datos y produce el HTML, sin consultar nada por su cuenta, y el navegador renderiza el resultado. La última parte importa: la vista no consulta, y si lo hiciera, el patrón estaría roto.

c) Por Qué MVC No Es Lo Mismo Que la Arquitectura en Capas. MVC y la arquitectura en capas no son lo mismo, y la diferencia está en el plano donde actúa cada uno. Las capas separan responsabilidades en la organización del código (presentación, negocio y acceso a datos), se concretan en niveles de despliegue y su alcance es todo el sistema; MVC, en cambio, separa responsabilidades dentro de una capa, ordena el interior de la presentación y divide el trabajo entre modelo, vista y controlador, que es la razón por la cual el libro lo llama patrón de

diseño y no arquitectura. En una frase: las capas responden a cómo se organiza la aplicación completa, y MVC a cómo fluye una petición dentro de esa organización.

d) Un Sistema Donde Conviven MVC y Tres Capas. Un sistema de matrículas universitario puede ser una arquitectura en tres capas (presentación, negocio y acceso a datos) desplegada en tres niveles y, al mismo tiempo, implementar MVC dentro de la capa de presentación: el controlador recibe la petición del navegador, el modelo es la proyección del estudiante y de sus cursos, y la vista es la plantilla que se renderiza. Las capas separan los niveles de responsabilidad del sistema y MVC ordena la conversación entre la interfaz, el estado de la vista y las reglas de presentación; cada uno resuelve un problema de un tamaño distinto y por eso conviven.

Pregunta 6 · Estilo, Patrón Arquitectónico y Patrón de Diseño

a) Definición de Cada Término. El libro distingue los tres términos por su alcance. El estilo arquitectónico es «the overarching structure of how the user interface and backend source code are organized ... and how that source code interacts with a datastore» (Richards y Ford, 2020), es decir, el nivel más general, donde se decide si el sistema será por capas, micronúcleo, microservicios u otro. El patrón arquitectónico es una estructura de diseño de nivel más bajo que forma soluciones concretas dentro de un estilo, «lower-level design structures that help form specific solutions within an architecture style» (Richards y Ford, 2020), como lograr alta escalabilidad o alto rendimiento sobre un conjunto de operaciones o entre servicios. El patrón de diseño es el de alcance más reducido, una solución con nombre para un problema de colaboración entre pocas clases u objetos; el libro lo ejemplifica con el Método Plantilla, «which allows parent classes to define workflow that can be overridden in child classes» (Richards y Ford, 2020).

b) Orden por Alcance y Su Justificación. El criterio para ordenarlos es cuánta parte del sistema afecta cada decisión: el estilo afecta al sistema completo y a su despliegue y por eso va primero; el patrón arquitectónico afecta a la organización de un subsistema o a la forma de resolver un problema dentro del estilo, y va segundo; y el patrón de diseño afecta a la relación entre unas pocas clases u objetos, y va al final. Ese orden lo respalda la relación de contención con que se definen en el literal a: el estilo marca el marco, el patrón arquitectónico forma soluciones dentro de ese marco y el patrón de diseño resuelve detalles de la implementación dentro de esas soluciones.

c) Un Ejemplo de Cada uno. Microservicios es un estilo: organiza todo el sistema como servicios desplegables de manera independiente, cada uno con su propia base de datos, y resuelve el problema de escalar y evolucionar cada parte por su cuenta. La mensajería asíncrona entre servicios es un patrón arquitectónico: desacopla operaciones dentro de un estilo distribuido y resuelve el problema de ganar rendimiento cuando un servicio depende de la respuesta de otro. El Método Plantilla es un patrón de diseño: define en la clase padre un flujo reutilizable que las clases hijas pueden sobrescribir, y resuelve el problema de variar un algoritmo sin duplicarlo.

d) Por Qué Confundir los Términos No Es Solo un Error de Vocabulario. Llamar patrón de diseño a una arquitectura completa, o al revés, no es solo un error de vocabulario sino una señal de que no se identificó el nivel de decisión en el que se trabaja. Los tres términos están ordenados por el alcance de la decisión, y confundirlos supone tratar una decisión que afecta a todo el sistema como un detalle de unas pocas clases o, a la inversa, atribuir a un detalle local un alcance de sistema. Decir que el sistema usa el patrón de diseño microservicios no transmite una decisión de estructura general, y decir que la arquitectura es el patrón Singleton confunde los dos niveles; quien no distingue el alcance de una decisión no puede discutirlo, y ese es justamente el tipo de conversación que se evalúa en esta materia.

7 · Puntos en Pareja

La guía organiza esta parte para dos integrantes como máximo. Este trabajo lo realiza una sola persona, de modo que los dos puntos se desarrollan de manera individual y no se consigna un segundo nombre. Las dos definiciones del primer punto y la decisión del segundo se apoyan en los libros físicos del Sistema de Bibliotecas cuyas portadas y contras se anexan al final (Universidad Simón Bolívar, 2026).

Punto Grupal 1 · Dos Autores, una Definición Propia

a) Las Dos Definiciones. Se consultaron dos libros físicos distintos del Sistema de Bibliotecas. El primero es *Fundamentals of Software Architecture*, de Richards y Ford (2020), donde la arquitectura se define como el conjunto que combina la estructura del sistema con las condiciones que este debe soportar: «software architecture consists of the structure of the system ... combined with architecture characteristics (“-ilities”) the system must support, architecture decisions, and finally design principles». El segundo es *Ingeniería de software: un enfoque práctico*, de Pressman y Maxim (2020), donde la arquitectura de software se presenta como el conjunto de decisiones significativas sobre la organización del sistema, la selección de los elementos estructurales y de sus interfaces, y el comportamiento que se deriva de la colaboración entre esos elementos. En ninguno de los dos casos se anota el número de página, porque no se fotografió la página exacta del ejemplar; la referencia se conserva con autor y año.

b) Contraste entre las Dos. La de Richards y Ford (2020) es formal y enumerativa: descompone la arquitectura en cuatro partes identificables, estructura, características, decisiones y principios, y habla de cualquier sistema, sin atarse a un dominio. La de Pressman y Maxim (2020) es más funcional: se centra en las decisiones significativas, en los elementos y en sus interfaces, y en el comportamiento que emerge de la colaboración. Lo que una incluye y la otra no: la primera suma los criterios de éxito, los atributos de calidad, como parte de la definición,

cosa que la segunda no hace explícita en esa formulación; la segunda subraya la idea de decisión significativa y de colaboración entre elementos, que en la primera queda repartida entre decisiones y principios. Coinciden en que la arquitectura no es código sino organización; se diferencian en que una es explícita sobre los criterios de éxito y la otra sobre la interdependencia de los elementos.

c) Definición Propia y Su Justificación. Tomando de Richards y Ford (2020) la estructura y los criterios de éxito, y de Pressman y Maxim (2020) la idea de decisión significativa y de colaboración, la definición propia queda así: la arquitectura de software es el conjunto de decisiones de organización que fija la estructura del sistema, define los criterios de éxito que este debe sostener y establece las reglas y los principios con que se construye, de modo que el comportamiento del conjunto dependa de la colaboración entre sus elementos y no de cada línea de código. De Richards y Ford se toma la estructura y los criterios de éxito porque acotan el alcance de la decisión y evitan reducir la arquitectura a un diagrama; de Pressman y Maxim se toma la idea de decisión significativa y de colaboración, porque explica por qué esas decisiones pesan sobre todo el sistema y no sobre un detalle local.

d) Datos de la Ficha y Fotografías. La ficha de biblioteca diligenciada se adjunta en el Anexo A, y las fotografías de la portada y de la contraportada de cada uno de los dos libros se incluyen en el Anexo C. El registro corresponde a la visita hecha dentro de la ventana del 14 al 21 de agosto de 2026, con los cuatro códigos de barras que constan en la ficha.

Punto Grupal 2 · Decidir y Sostener la Decisión

El caso que se resuelve es el siguiente: la facultad necesita un sistema web para reservar los equipos de los laboratorios de cómputo, con unos 1.200 estudiantes y 40 docentes; las reservas se concentran en las dos primeras semanas de cada semestre y el resto del tiempo el uso es bajo; el equipo de desarrollo son dos personas; la universidad ya tiene servidores propios y no

quiere pagar servicios en la nube; y el sistema debe integrarse con el sistema de autenticación institucional existente.

a) Atributos de Calidad Priorizados. Como se muestra en la Tabla 5, los atributos se ordenan según su peso en el enunciado.

Tabla 5

Atributos de Calidad del Caso, Ordenados por Prioridad

Atributo	Prioridad	Justificación con datos del enunciado
Escalabilidad	Alta	Las reservas se concentran en las dos primeras semanas de cada semestre y son 1.200 estudiantes; la demanda de ese lapso es muy superior a la del resto del año.
Disponibilidad	Alta	Una caída justo en las dos semanas de reservas afecta a estudiantes y docentes; fuera de ese lapso la exigencia baja.
Seguridad	Alta	Debe integrarse con la autenticación institucional existente y distinguir roles entre estudiante, docente y administrador.
Mantenibilidad	Alta	El equipo de desarrollo son dos personas: menos piezas significa menos riesgo de errores y menos trabajo de operación.
Economía	Alta	La universidad tiene servidores propios y no quiere pagar servicios en la nube, así que la solución debe correr en hardware existente.
Usabilidad	Media	Son 1.200 estudiantes reservando; la interfaz debe ser simple, pero el pico acotado no justifica una aplicación compleja.

Nota. El orden responde al peso de los datos del enunciado: el pico concentrado manda sobre la escalabilidad y la disponibilidad, y el equipo pequeño y la restricción de no pagar nube mandan sobre la mantenibilidad y la economía.

b) Dos Alternativas de Arquitectura. Como se muestra en la Tabla 6, cada alternativa favorece un atributo y sacrifica otro.

Tabla 6

Dos Alternativas de Arquitectura para el Caso

Alternativa	En qué consiste	Qué atributo favorece	Qué sacrifica	Cuándo sería mala decisión
1. Monolito por capas	Aplicación en capas desplegada en un servidor propio, con la base de datos relacional en el mismo servidor y una interfaz web ligera.	Simplicidad, mantenibilidad y economía.	Escalabilidad horizontal y tolerancia a fallos: hay un solo punto de falla.	Si el pico de dos semanas supera la capacidad de un solo servidor o si la facultad exige que siga arriba aun cayendo esa máquina.
2. Dos niveles	La misma lógica por capas desplegada en dos máquinas,	Escalabilidad y disponibilidad: la capa de aplicación se puede duplicar y una máquina	Más infraestructura (un servidor más) y más	Si el presupuesto de hardware es mínimo, si el equipo de dos personas no puede

Alternativa	En qué consiste	Qué atributo favorece	Qué sacrifica	Cuándo sería mala decisión
	servidor de aplicación y servidor de base de datos, con caché para el pico.	puede asumir si la otra falla.	complejidad operativa.	operar dos servidores, o si el uso del año es tan bajo que la segunda máquina queda ociosa casi todo el tiempo.

Nota. Se descartan las arquitecturas distribuidas tipo microservicios, por ser la opción que la guía señala como sobre-diseño para un equipo pequeño y un dominio de pocos procesos.

c) Elección y Defensa. Se elige la alternativa 1, el monolito por capas en un servidor propio. Con dos personas y sin presupuesto para la nube, la prioridad es construir y operar un sistema confiable y barato; el estilo por capas es sencillo, conocido y de bajo costo, y el pico, aunque alto, es predecible y acotado a dos semanas. En lugar de repartir el sistema, se escala ese único despliegue durante la ventana de reservas: más memoria y procesador, una caché para las consultas repetidas y, si la demanda lo exige, una copia de la aplicación en una segunda máquina solo en el pico. La decisión reconoce sus renunciaciones: la escalabilidad es vertical y limitada, hay un único punto de fallo y una caída de esa máquina afecta a todos, por lo que el mantenimiento debe programarse fuera de las dos semanas críticas. Si el pico fuera mayor o constante, la salida sería adoptar lo que ofrece la alternativa 2, separar los niveles y duplicar la aplicación, pero la infraestructura y el costo de esa opción no se justifican hoy para un equipo de dos personas.

d) Decisión del Equipo Frente a la Herramienta. La misma consulta se hizo a la herramienta `opencode`, modelo `deepseek-v4-flash`, el 23 de agosto de 2026. La respuesta se transcribe literal a continuación, con los pasajes más extensos recortados y marcados con [...]:

Claro, con gusto te ayudo a diseñar esa arquitectura. Para tu caso yo elegiría una arquitectura de **monolito modular** desplegada en los servidores propios, con una separación clara en componentes lógicos y un frontend que se integra con SSO institucional. [...] **Atributos de calidad que tendría en cuenta**. 1. **Coste de infraestructura**: la universidad ya tiene servidores, así que la solución no debe exigir servicios en la nube ni licencias costosas. 2. **Simplicidad y mantenibilidad**: el equipo es de dos personas, así que la complejidad técnica debe ser baja. 3. **Escalabilidad de las reservas**: el pico de carga es enorme en las dos primeras semanas del semestre y casi nulo el resto del año. 4. **Disponibilidad**: el sistema debe estar arriba sobre todo en época de reservas. 5. **Seguridad e integración con identidad institucional**: debe conectar con el SSO existente sin duplicar credenciales y respetar roles (estudiante, docente, administrador). 6. **Concordancia de datos**. 7. **Portabilidad y deuda técnica**. [...] **Alternativa A: Monolito modular (backend monolítico + frontend SPA)** [...] El frontend es una aplicación de página única que consume una API REST que expone el mismo backend. Se despliega en el servidor propio con una base de datos relacional (PostgreSQL/MySQL). [...] **Alternativa B: Microservicios distribuidos** Cada dominio (usuarios, equipos, reservas, notificaciones) es un servicio independiente, cada uno con su propia base de datos [...] Se despliega en contenedores (Docker/Kubernetes) sobre el clúster propio. [...] **Mi elección y el motivo**. Elegiría la **Alternativa A, el monolito modular**. La razón principal es que tu contexto pide pocas piezas y mucho control: con un equipo de dos personas, los microservicios añaden una complejidad operativa [...] que no se traduce en beneficio real. El pico de carga, aunque alto, es de naturaleza predecible y acotada, y puede resolverse mucho más fácil escalando el monolito: subir instancias adicionales del mismo backend o añadir caché y una

cola de fondo para los procesos de reserva, incluso habilitando un servidor extra solo durante el pico. [...] **Resumen**: por costo, simplicidad, facilidad de integración con el SSO y capacidad de absorber el pico con la infraestructura existente, la arquitectura recomendada es un **monolito modular** desplegado en los servidores propios, con base de datos relacional y un frontend SPA, acompañado de caché y de una cola de procesamiento para los momentos de mayor demanda.

La decisión del equipo coincide con la herramienta en lo esencial, porque ambas descartan los microservicios y se quedan con un monolito. Difieren, sin embargo, en un punto concreto. La herramienta recomienda sostener el pico «subiendo instancias adicionales del mismo backend o añadiendo caché y una cola de fondo, incluso habilitando un servidor extra solo durante el pico», y propone a la vez un frontend de aplicación de página única (SPA) con caché y cola de procesamiento. La decisión del equipo es más austera: mantiene una interfaz renderizada en el servidor y solo agrega una segunda máquina en el pico si de verdad se necesita, sin incorporar cola de fondo ni el frontend SPA, porque el equipo es de dos personas y la operación adicional que la herramienta da por hecha no es gratuita. La herramienta también propone, como alternativa, microservicios «en contenedores sobre el clúster propio», lo que exige conocimiento de operación que dos personas no pueden sostener. Por eso se considera que la decisión del equipo es más ajustada al contexto, que es justo lo que la guía marca como criterio para elegir una arquitectura frente a otra (Universidad Simón Bolívar, 2026).

Uso de IA

Esta sección declara el uso de inteligencia artificial en todo el trabajo, como exige la guía. No sustituye al ejercicio de la sección 4, que es un análisis aparte (Universidad Simón Bolívar, 2026).

Tabla 7

Declaración del Uso de Inteligencia Artificial

Campo	Qué se declara
Herramienta	opencode, modelo deepseek-v4-flash. No se identificó la versión. Consultas realizadas el 23 de agosto de 2026.
Para qué	Redactar y contrastar los prompts de la sección 4, obtener respuestas para analizarlas, contrastar alternativas en el punto grupal 2 y revisar la redacción propia.
Prompt principal	Se transcribe a continuación, tal como se envió a la herramienta. Es el prompt de la sección 4.2c.
Qué conservó	La idea de que la arquitectura combina estructura, características, decisiones y principios; el descarte de los microservicios para un equipo de dos personas; y la estructura por capas como punto de partida.
Qué corrigió	La herramienta añadió un frontend de página única, una cola de procesamiento y un servidor extra para el pico, y propuso tecnologías concretas y un clúster para los microservicios. Se corrigió manteniendo una interfaz renderizada en el servidor y sin cola ni clúster, porque el equipo de dos personas y la restricción de no pagar servicios en la nube no lo justifican; la verificación se hizo contra el libro.

Prompt principal transcrito.

[ROL] Actúas como un ingeniero de software con experiencia en despliegue y operación de aplicaciones web.

[TAREA] Explicame por qué en una aplicación web se separa el servidor web del servidor de base de datos y qué implica esa separación para la conectividad entre ambos.

[CONTEXTO] Soy estudiante de séptimo semestre de Ingeniería de Sistemas. He hecho aplicaciones web que corren en un solo servidor, con la base de datos en el mismo equipo, y no entiendo por qué en entornos de producción los separan.

[FORMATO] Empieza con una respuesta directa de máximo tres líneas.

Después una tabla de tres columnas: qué cambia con la separación, qué gana, qué cuesta. Termina explicando el papel de la conexión entre ambos servidores.

[RESTRICCIONES] No nombres tecnologías comerciales concretas. Máximo 300 palabras. Distingue entre lo que es consenso de la disciplina y lo que es tu interpretación, marcando con «(mi interpretación)».

[VERIFICACIÓN] Cierra diciendo qué afirmaciones debería yo confirmar en un libro antes de darlas por buenas y qué información adicional necesitarías tú para responder con más precisión.

Conclusiones

El ejercicio de manejo del prompt dejó una primera conclusión: un encargo bien formulado produce una respuesta aprovechable, y la diferencia no está en pedir más sino en precisar el rol, la tarea, el contexto, el formato, las restricciones y la verificación. En este caso la

tarea fue el elemento que más cambió el resultado y el rol casi no aportó, lo que confirma que no todos los elementos pesan igual en todos los encargos (Universidad Simón Bolívar, 2026).

La segunda conclusión es sobre la materia. Los conceptos de la unidad se encadenan y no se entienden por separado: los atributos de calidad condicionan la arquitectura, la arquitectura se concreta en niveles y en capas, y el patrón Modelo-Vista-Controlador organiza el interior de una de esas capas. Confundir nivel con capa, o MVC con capas, lleva a dimensionar mal la infraestructura y a sobre-diseñar sistemas que un equipo pequeño no puede operar. La inteligencia artificial sirve para contrastar, para encontrar pistas y para revisar lo ya escrito, pero no para decidir ni para sostener una decisión; esa parte, que es justo lo que se evalúa, no se delega.

Referencias

Bruegge, B., y Dutoit, A. H. (2002). *Ingeniería de software orientado a objetos*. Prentice Hall.

Pressman, R. S., y Maxim, B. R. (2020). *Ingeniería de software: Un enfoque práctico* (9.^a ed.). McGraw-Hill.

Richards, M., y Ford, N. (2020). *Fundamentals of software architecture: An engineering approach*. O'Reilly Media.

Sommerville, I. (2005). *Ingeniería del software* (7.^a ed.). Pearson Addison-Wesley.

Universidad Simón Bolívar, Facultad de Ingenierías. (2026). *La IA como apoyo y el manejo del prompt* [Guía de la actividad. Curso Arquitectura de Software, código 5111, docente Oswaldo Enrique Laguado Aguilar].

Anexos

Anexo A

Ficha de Biblioteca Diligenciada



SISTEMA DE BIBLIOTECAS

REGISTRO DE CONSULTA EN SALA

	PROGRAMA ACADÉMICO	CÓDIGO ESTUDIANTEL Y/O PROFESORES	CÓDIGO DE BARRAS LIBROS CONSULTADOS
1	Inq. Sistemas	202413228925	018634
2	"	"	005133
3	"	"	006961
4	"	"	018642
5			
6			
7			
8			
9			
10			
11			
12			
13			
14			
15			
16			
17			
18			
19			
20			

Nota. El registro corresponde a la consulta en sala realizada en el Sistema de Bibliotecas. Consta el programa académico (Ing. Sistemas), el código estudiantil 202413228925 y cuatro códigos de barras de los libros consultados: 018634, 005133, 006969 y 018645.

Anexo B

Registro de Asistencia a la Biblioteca

La asistencia quedó registrada en el control de ingreso de la biblioteca durante la visita a la biblioteca Anita Consuegra, dentro de la ventana del 14 al 21 de agosto de 2026, y la consulta en sala consta en la ficha del Anexo A.

Anexo C

Fotografías de la Portada y de la Contraportada de los Libros Consultados

Se incluyen la portada y la contraportada de cada ejemplar consultado, conforme al registro del Sistema de Bibliotecas; los cuatro libros corresponden a los códigos de barras de la ficha y sus fotografías se presentan en las Figuras 1 a 4.

Figura 1

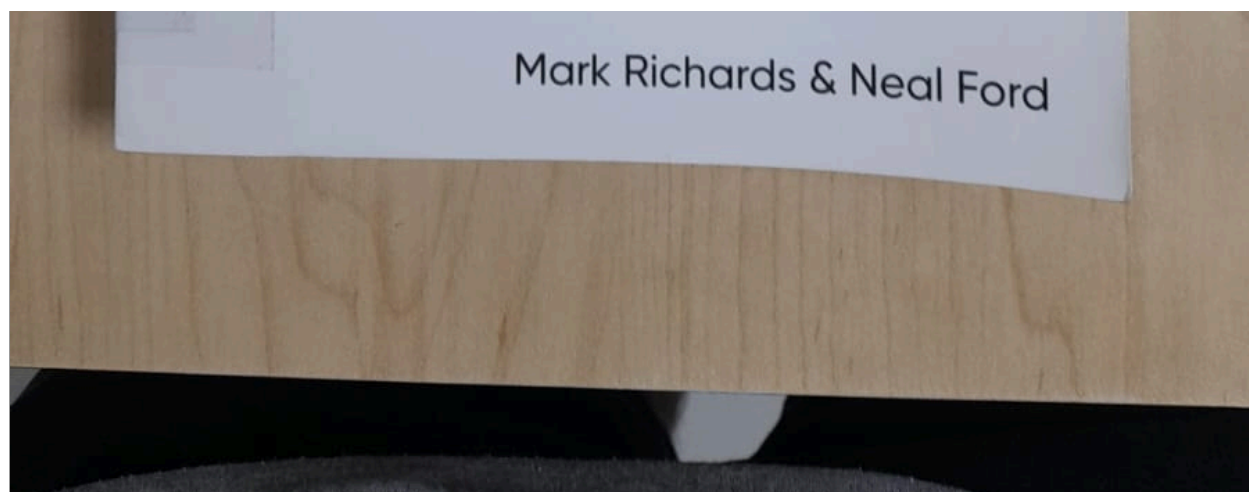
Portada y Contraportada de Fundamentals of Software Architecture

O'REILLY®



Fundamentals of Software Architecture

An Engineering Approach



Fundamentals of Software Architecture

An Engineering Approach

Mark Richards and Neal Ford

Beijing • Boston • Farnham • Sebastopol • Tokyo

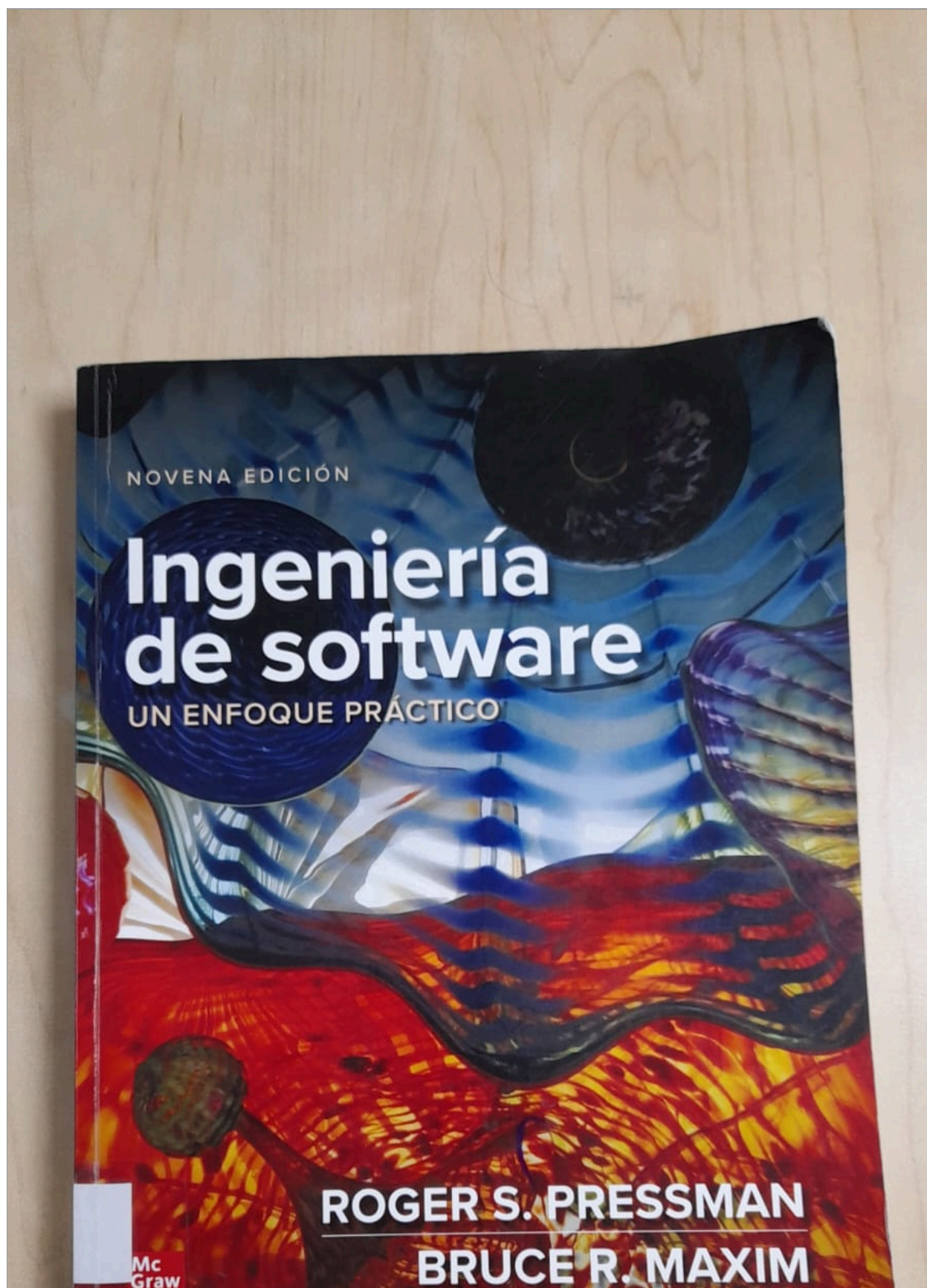
O'REILLY®



Nota. Richards, M., y Ford, N. (2020). *Fundamentals of software architecture: An engineering approach*. O'Reilly Media.

Figura 2

Portada y Contraportada de Ingeniería de software: un enfoque práctico





NOVENA EDICIÓN

Ingeniería de software

Un enfoque práctico

Roger S. Pressman, Ph.D.

Bruce R. Maxim, Ph.D.

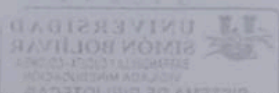
Revisión técnica

Luis Antonio Medellín Serna

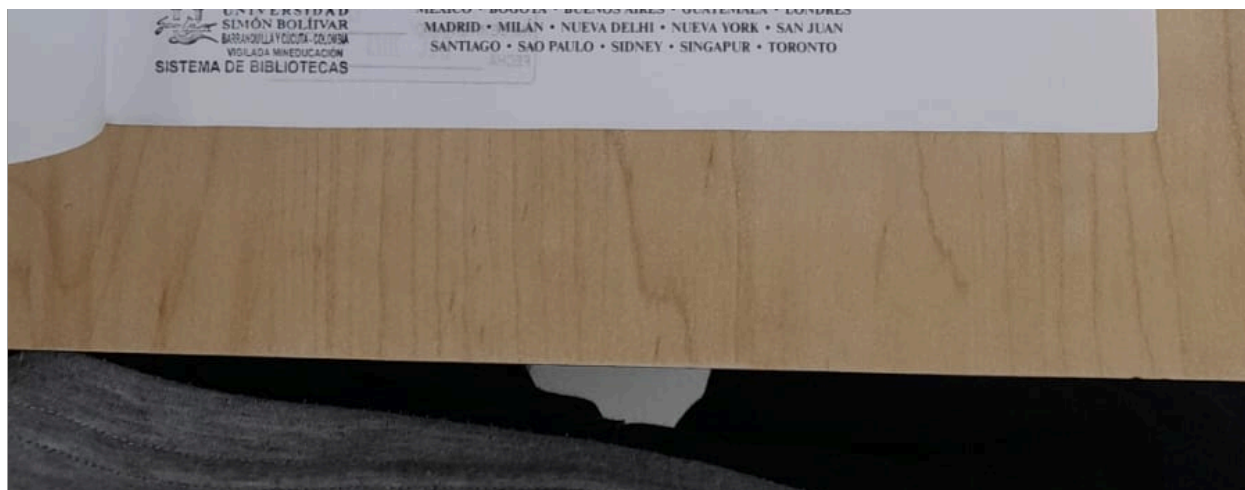
Departamento de Ciencias Computacionales

Centro Universitario de Ciencias Exactas e Ingenierías (CUCEI)

Universidad de Guadalajara, México



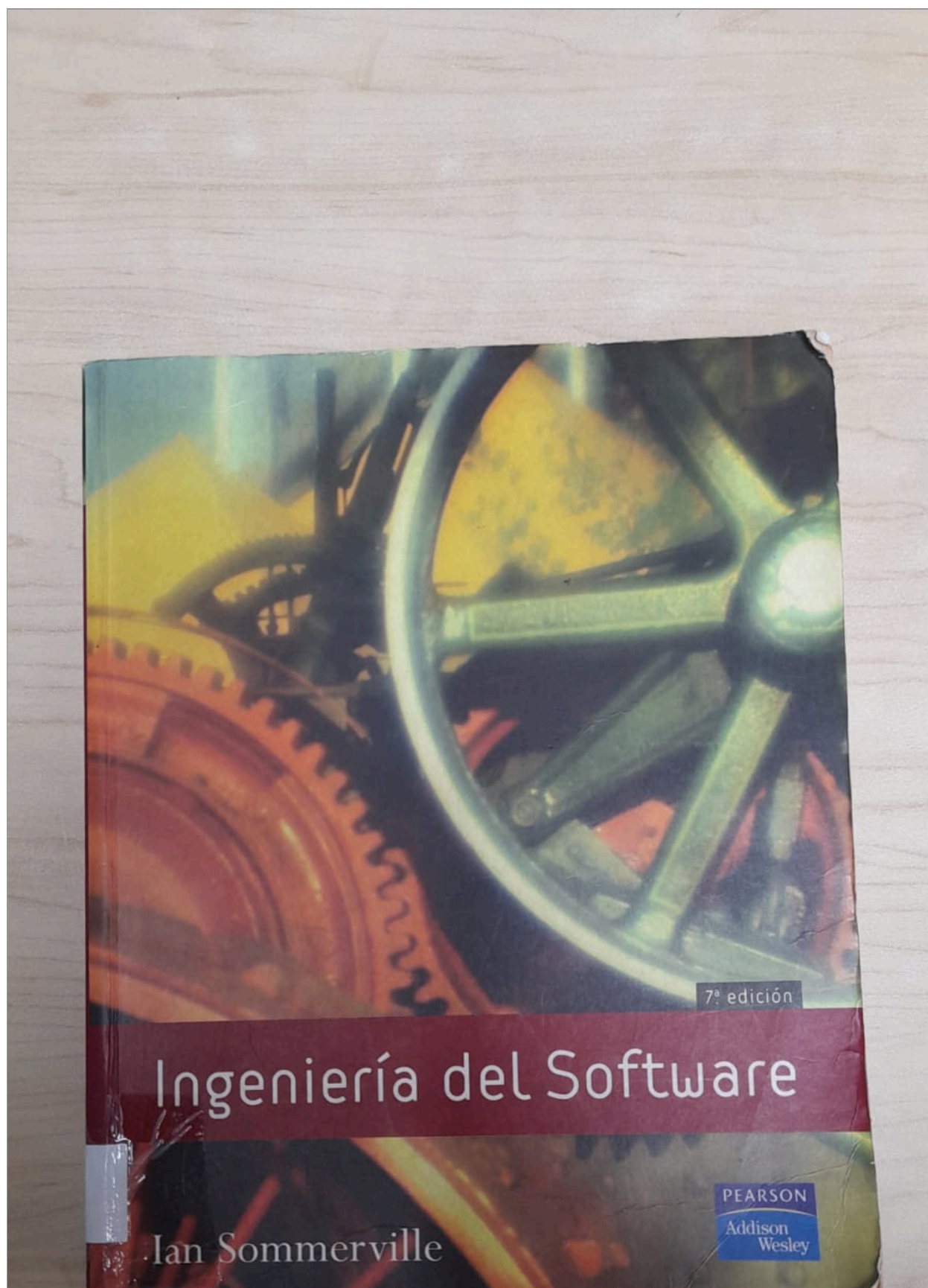
**Mc
Graw
Hill**

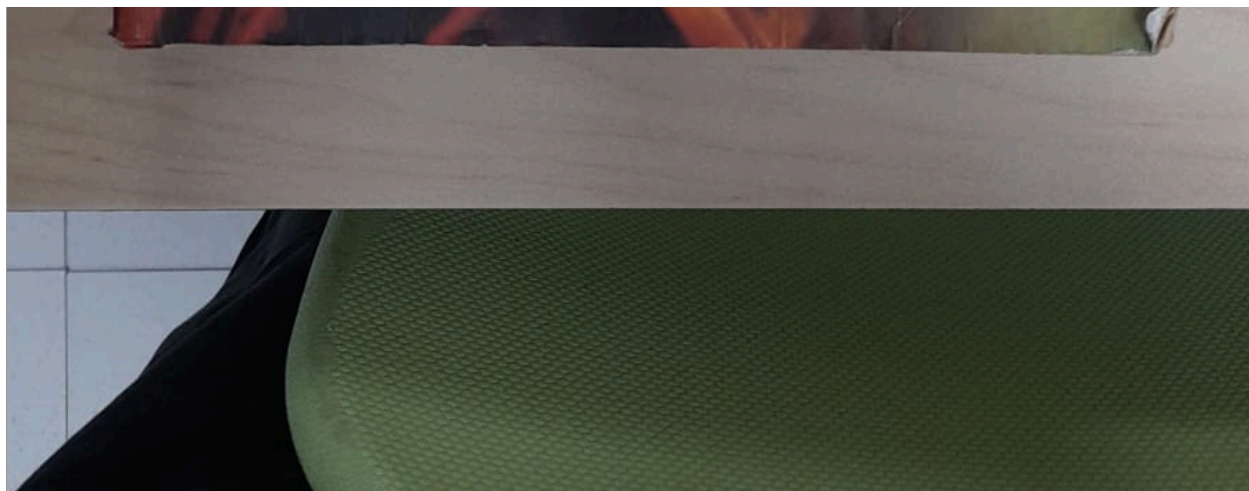


Nota. Pressman, R. S., y Maxim, B. R. (2020). *Ingeniería de software: Un enfoque práctico* (9.^a ed.). McGraw-Hill.

Figura 3

Portada y Contraportada de Ingeniería del software





Ingeniería del software

Séptima edición

IAN SOMMERVILLE

Traducción

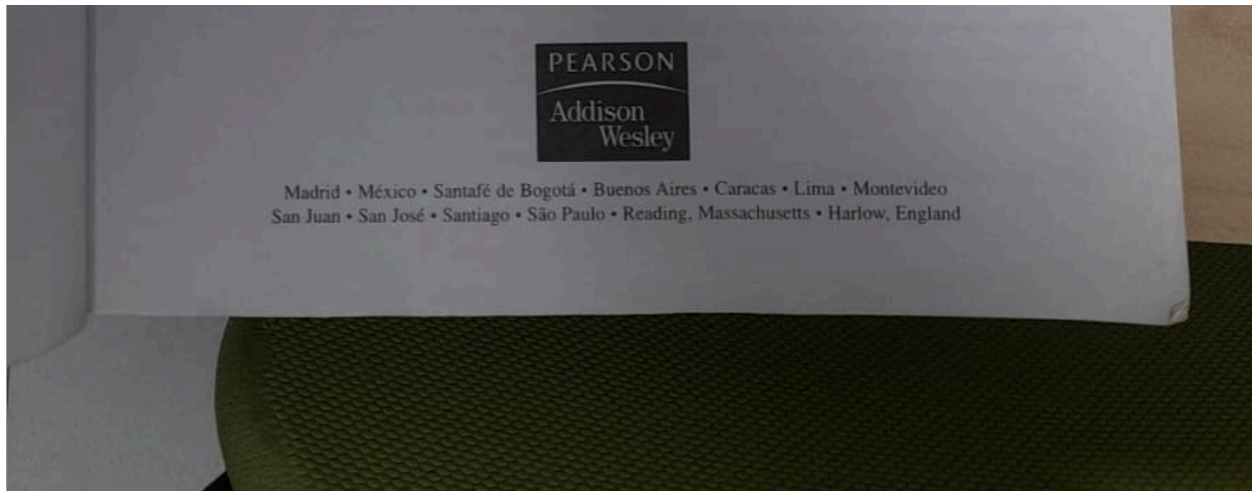
María Isabel Alfonso Galipienso

Antonio Botía Martínez

Francisco Mora Lizán

José Pascual Trigueros Jover

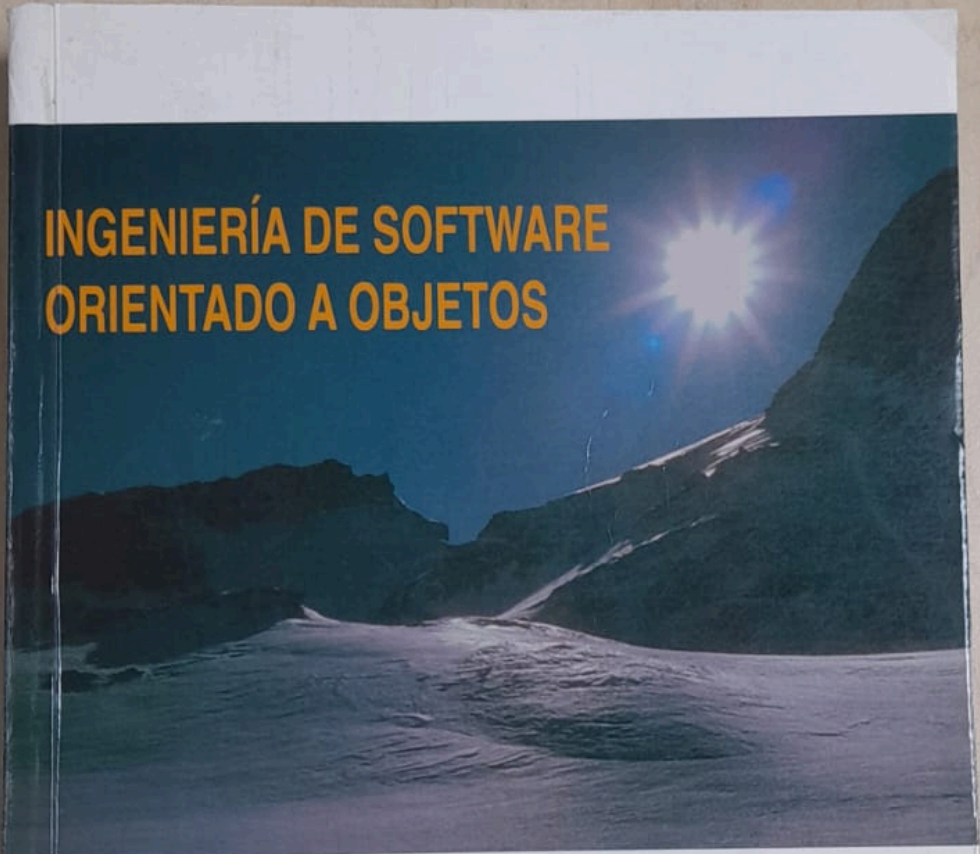
*Departamento Ciencia de la Computación e Inteligencia Artificial
Universidad de Alicante*



Nota. Sommerville, I. (2005). *Ingeniería del software* (7.^a ed.). Pearson Addison-Wesley.

Figura 4

Portada y Contraportada de Ingeniería de software orientado a objetos



INGENIERÍA DE SOFTWARE ORIENTADO A OBJETOS

BERND BRUEGGE • ALLEN H. DUTOIT

Prentice
Hall



Ingeniería de software orientado a objetos

Bernd Bruegge

Technical University of Munich
Munich, Germany

Allen H. Dutoit

Carnegie Mellon University
Pittsburgh, PA, United States

TRADUCCIÓN:

Sergio Luis María Ruiz Faudón

Ingeniero Químico, Facultad de Ciencias Químicas, Universidad Veracruzana, campus Orizaba

REVISIÓN TÉCNICA:

Rafael Gamboa Hiales

Maestro en Ingeniería, Universidad Politécnica de Madrid

Departamento Académico de Computación, Instituto Tecnológico Autónomo de México

Martha Rosa Cordero López

Maestra en Ciencias, Centro de Investigación y de Estudios Avanzados, Instituto Politécnico

Nacional

Escuela Superior de Cómputo

Instituto Politécnico Nacional

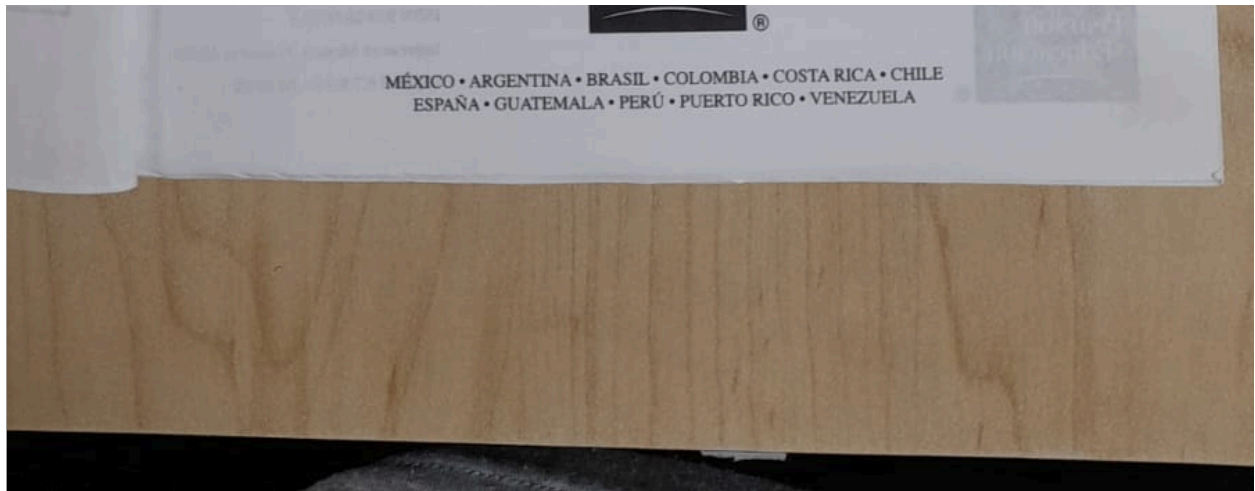
Marco Antonio Dorantes González

Maestro en Ciencias, Centro de Investigación y de Estudios Avanzados, Instituto Politécnico

Nacional

Escuela Superior de Cómputo

Instituto Politécnico Nacional



Nota. Bruegge, B., y Dutoit, A. H. (2002). *Ingeniería de software orientado a objetos*. Prentice Hall.